

Reordering Buffer

Optimal Online Algorithm

Noa Avigdor-Elgrabli & Yuval Rabani



The Problem

Input: n colored items

Output: a permutation,
using a buffer of capacity k

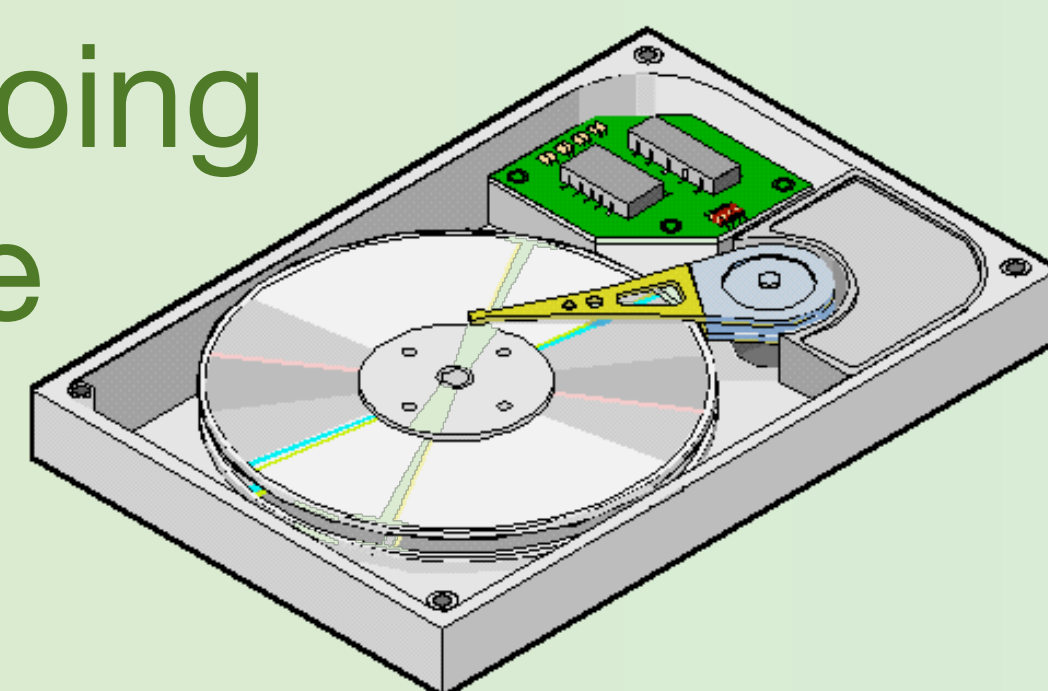
Objective: minimize the
number of output color changes



Buffer ($k=3$)

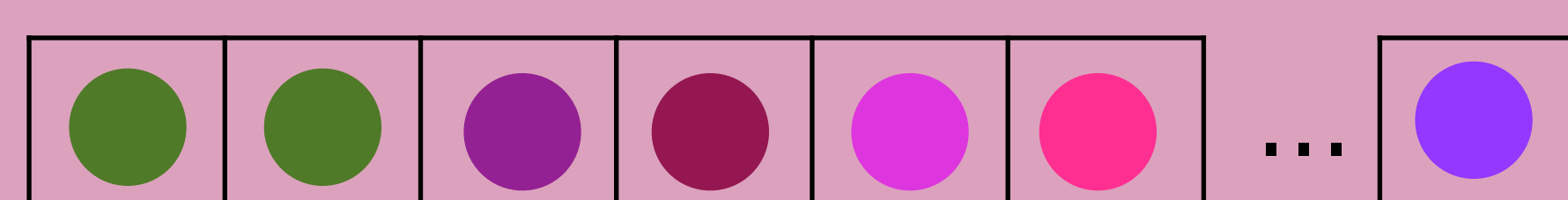
Motivations

Disk Scheduling : A reordering buffer can be used to rearrange the incoming sequence of accesses to the hard disk in a way that minimizes the movements of the head to the respective cylinders and by doing so improves the latency of the access.



Greedy Doesn't Work

Input sequence: ● ● ● ● ● ● ● ... ●



Buffer

Greedy: ● ● ● ● ● ● ● ... ● ● ● ● ● ● ● ... ● $\Omega(k+n)$

Opt: ● ● ● ● ● ● ● ... ● ● ● ● ● ● ● ... ● $O(k+n/k)$

*Every algorithm is $O(2k-1)$ -competitive

Previous Work

- Offline: NP-hard [?]
 $O(1)$ -approx [AR13]
- Online: $O(\sqrt{\log k})$ algorithm.
 $\Omega(\sqrt{\log k / \log \log k})$ (det.)
 $\Omega(\log \log k)$ (rand.) [A?]

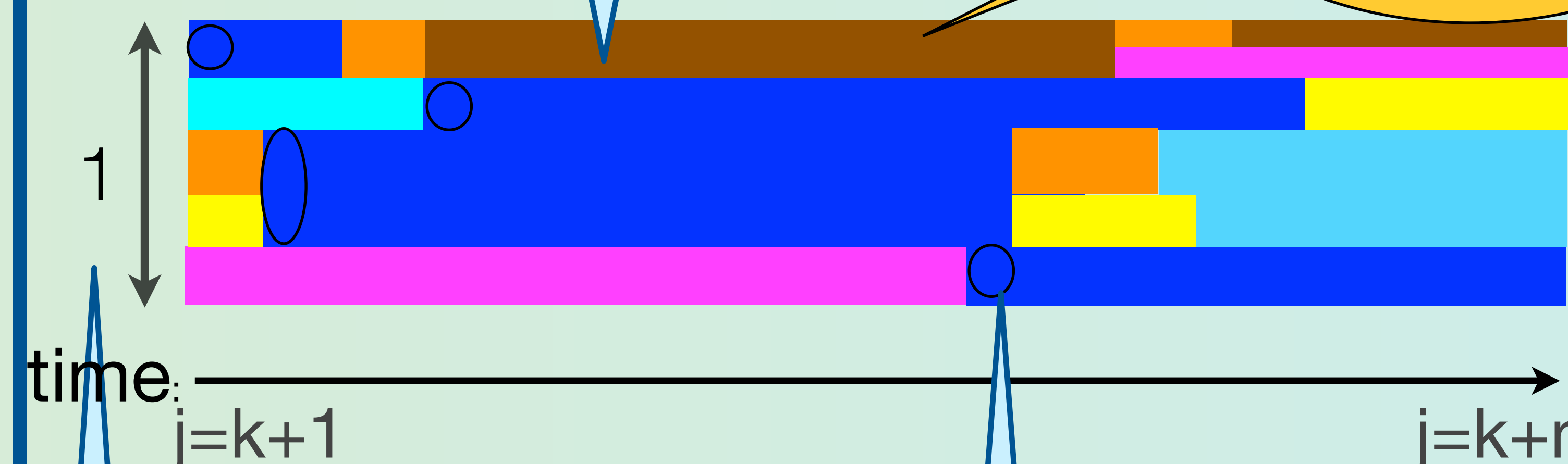
Our Result

$O(\log \log k)$ - competitive
randomized algorithm

The LP Relaxation

A fractional packing of
sequences of consecutive
items from the same color

Goal:
minimize the sum
of sequences
weight

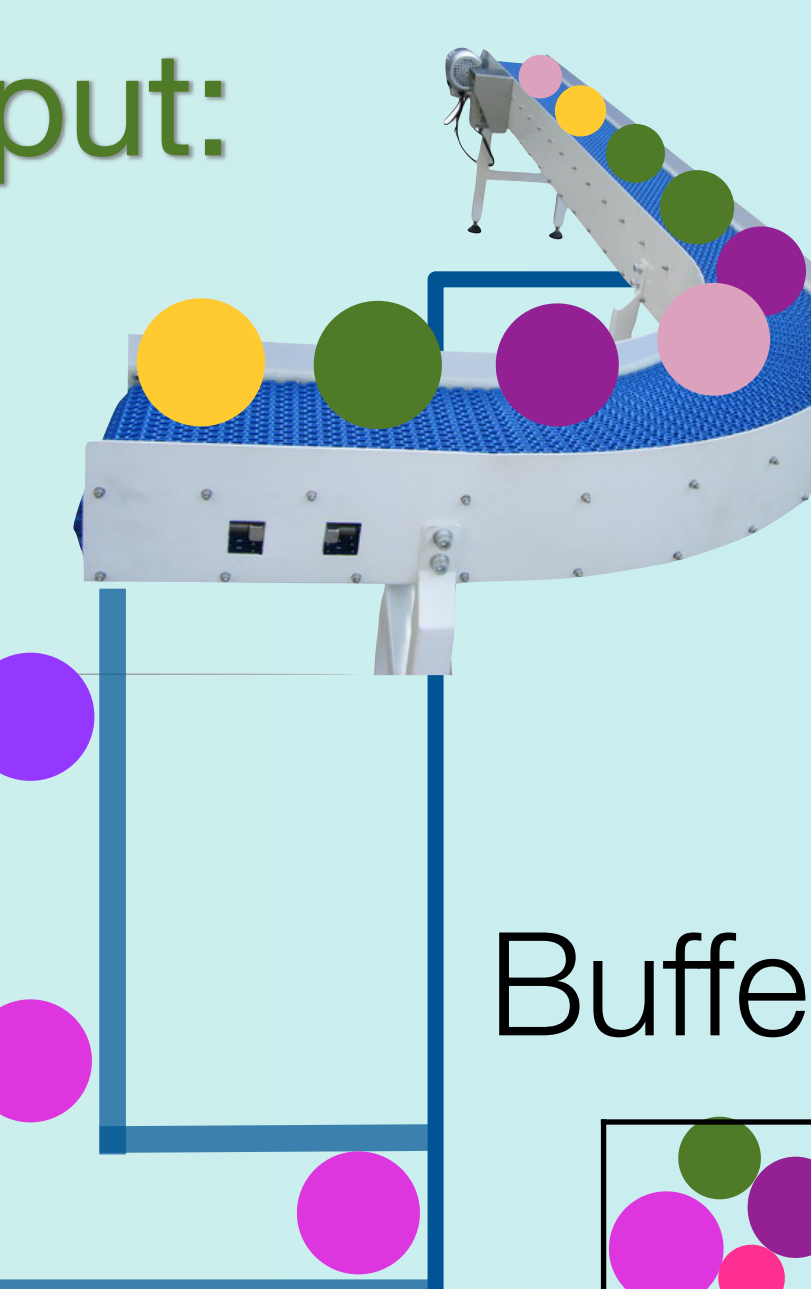


At every time step a
weight of 1 sequences
is scheduled

Every item is part
of a total weight
of 1 sequences

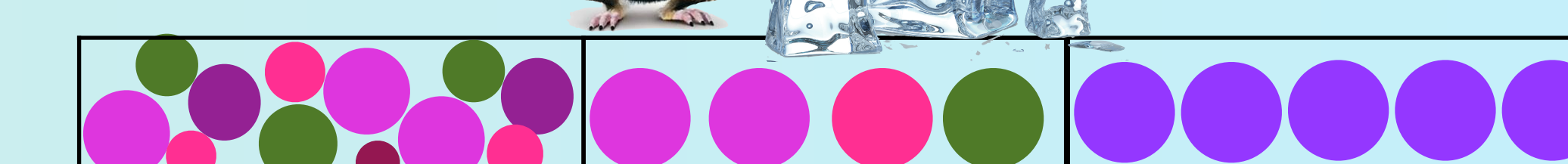
The Algorithm

Input:



Building the LP Solution Online

Buffer:



Fractional
Small blocks

Frozen
May defrost
as large or small

Integral
Large blocks

Online
primal dual

For fractional
small blocks

Integral
dual fitting

For integral
large blocks

Online Rounding

Using the offline
algorithm ideas [AR13]

Randomness

Output:

